

INFORME N°7

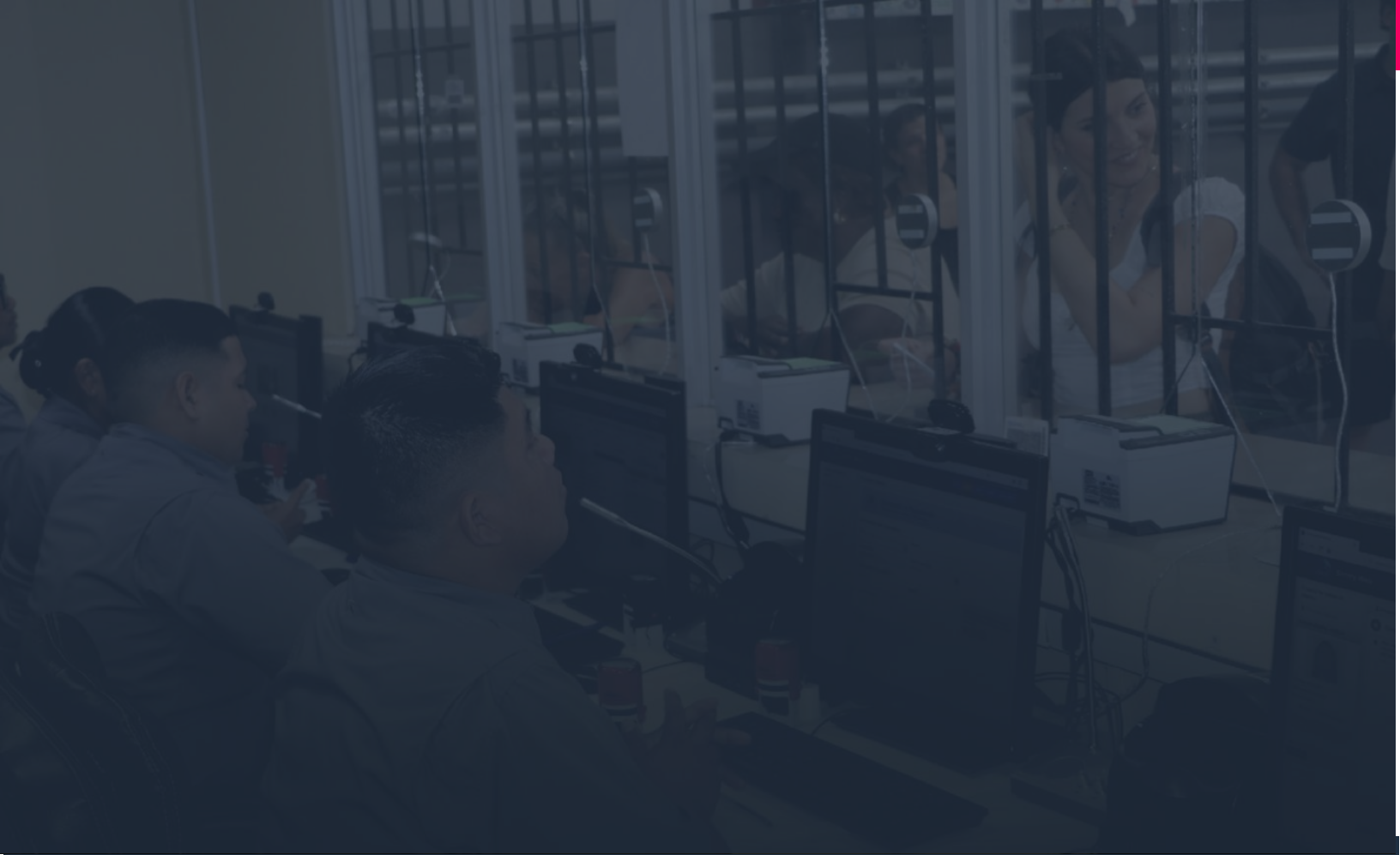
Reporte de desarrollo back-end, integraciones Rest API para el prototipo

EVALUACIÓN DE BASES DE DATOS Y ARQUITECTURA DE
SOLUCIÓN DE PROCESOS PARA EL SERVICIO NACIONAL DE
PANAMÁ



Contenidos

I. RESUMEN EJECUTIVO	5
II. OBJETIVOS	7
▪ OBJETIVO GENERAL DE LA CONSULTORÍA	7
▪ OBJETIVOS DE ESTE INFORME	7
III. DESARROLLO BACK-END	9
▪ DESARROLLO DE MODELO DE DATOS	10
▪ CONFIGURACIÓN E INTEGRACIÓN BBDD	12
▪ PRUEBAS UNITARIAS	16
▪ PRUEBAS INTEGRALES	18
▪ PROPUESTA DE CAPACITACIÓN Y DOCUMENTACIÓN	20
IV. INTEGRACIONES Y APIS	22
▪ DESARROLLO DE COMPONENTES DE LOS PRODUCTOS BACK-END	22
▪ PRUEBAS UNITARIAS	28
▪ PRUEBAS INTEGRALES	30
▪ PROPUESTA DE CAPACITACIÓN Y DOCUMENTACIÓN	34



01

Resumen Ejecutivo

I. RESUMEN EJECUTIVO

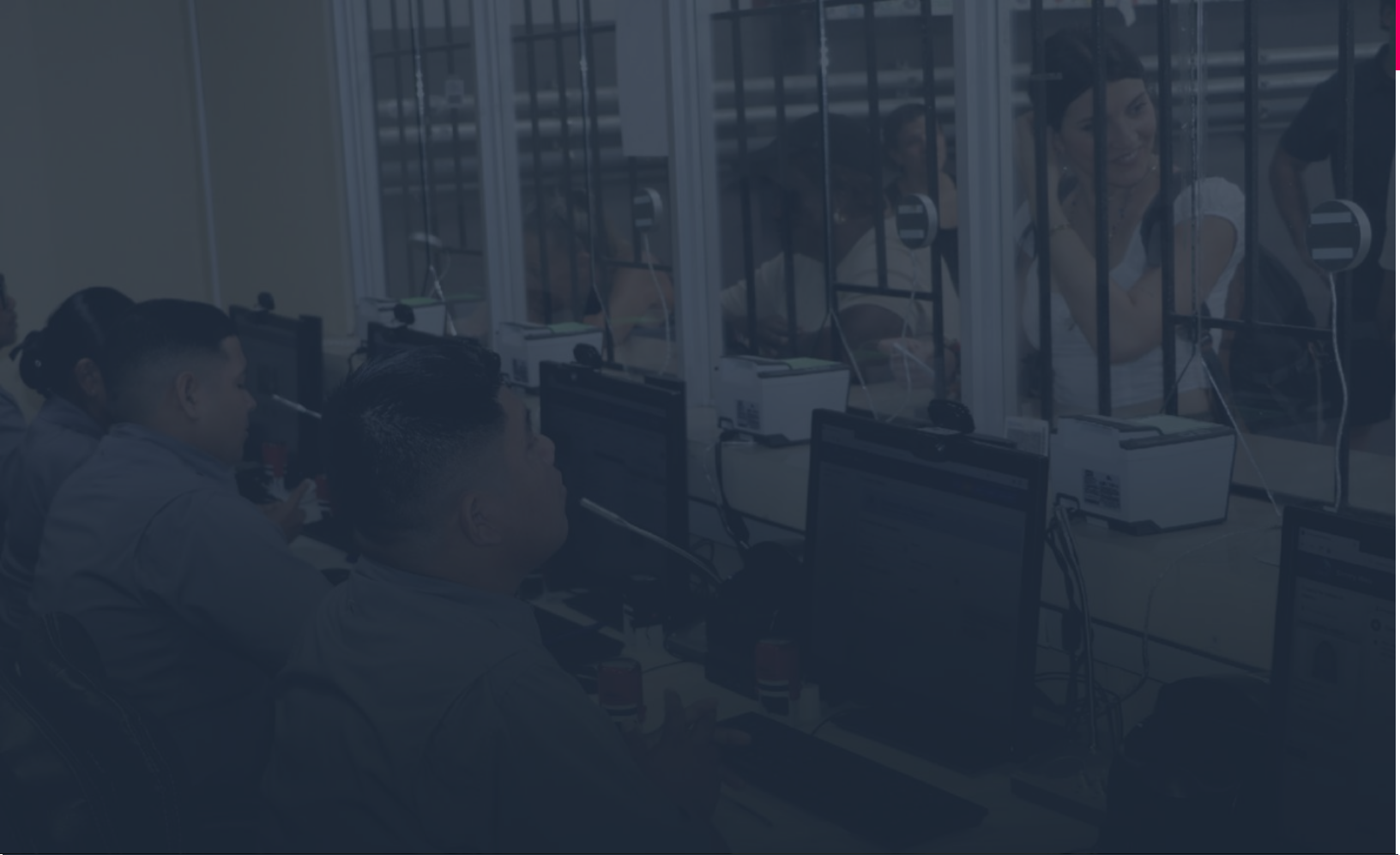
El presente documento constituye el **reporte técnico** sobre el desarrollo del back-end y las integraciones API REST para el prototipo de plataforma institucional del Servicio Nacional de Migración de Panamá (SNM).

El objetivo central del desarrollo fue construir una arquitectura modular y escalable mediante la implementación de un motor de procesos low-code capaz de interpretar estructuras JSON y notación BPMN 2.0, permitiendo configurar y modificar flujos de trabajo sin necesidad de alterar el código fuente. Esta aproximación estratégica se fundamenta en el modelado BPMN 2.0 de los procesos previamente levantados, garantizando alineación entre los requisitos funcionales y la implementación técnica.

Entre los componentes clave desarrollados en el back-end se encuentran: un motor de workflow configurable y reutilizable, un sistema de gestión de etapas procesales, lógica especializada para carga y revisión documental, y un módulo de validación mediante reconocimiento óptico de caracteres (OCR). El informe detalla exhaustivamente el desarrollo del modelo de datos relacional, así como la configuración e integración de la base de datos Microsoft SQL Server con la aplicación.

Se reportan los resultados de las pruebas unitarias e integrales realizadas tanto para los componentes del back-end como para las interfaces de programación de aplicaciones (APIs), asegurando la funcionalidad, robustez y confiabilidad del sistema. Asimismo, se incluyen propuestas concretas de capacitación y documentación técnica destinadas a asegurar la transferencia efectiva de conocimiento y la sostenibilidad a largo plazo del proyecto.

La arquitectura implementada en modalidad local (on-premise) resguarda adecuadamente la seguridad y confidencialidad de datos sensibles de los solicitantes. El diseño modular adoptado facilita significativamente el mantenimiento continuo y la evolución futura del sistema por parte de los equipos técnicos internos del SNM.



02

Objetivos

II. OBJETIVOS

La presente sección tiene por propósito relevar el objetivo general, los requerimientos específicos y el alcance del servicio ofertado.

OBJETIVO GENERAL DE LA CONSULTORÍA



El objetivo de este proyecto es apoyar al Servicio Nacional panameño en: (i) evaluar la calidad de datos contenidos en las múltiples bases de datos de SNM; (ii) realizar una revisión del levantamiento de cuatro (4) trámites migratorios de alto volumen dentro del Servicio Nacional de Migración; (iii) crear un prototipo funcional de uno de los tramites analizados.

OBJETIVOS DE ESTE INFORME



Reporte de desarrollo back-end



Integraciones Rest API para el prototipo.



03

Desarrollo Back-End

III. DESARROLLO BACK-END

El diseño y desarrollo del componente back-end del prototipo se fundamenta en el modelado exhaustivo de procesos mediante notación BPMN 2.0, el cual se realizó durante las fases previas de levantamiento de los cuatro (4) trámites migratorios priorizados, priorizando el proceso PPSH. Este enfoque garantiza la trazabilidad completa entre los requisitos funcionales documentados y la implementación técnica ejecutada.

La propuesta arquitectónica se centra en un motor de workflow genérico, modular y parametrizado capaz de interpretar estructuras en formato JSON que representan las definiciones de procesos, etapas, tareas y reglas de negocio. Esta aproximación low-code permite que el sistema ejecute múltiples procesos heterogéneos de forma simultánea sin requerir modificaciones en el código fuente, simplemente ajustando las configuraciones JSON asociadas a cada trámite.

La capacidad de trazabilidad integral constituye un atributo diferenciador del motor implementado, permitiendo el registro detallado de cada transición de estado, acción ejecutada, usuario responsable y timestamp asociado a lo largo de todo el ciclo de vida de una solicitud. Esta funcionalidad resulta fundamental tanto para auditoría como para la generación de indicadores de gestión y desempeño.

El motor de workflow establece una base tecnológica versátil que no solamente soporta los cuatro (4) trámites inicialmente modelados, sino que sienta las bases para la incorporación futura de procesos adicionales mediante la simple configuración de nuevas definiciones JSON, sin necesidad de redesarrollo o modificaciones arquitectónicas significativas. Esta característica resulta esencial para la escalabilidad y sostenibilidad a largo plazo de la plataforma institucional del SNM.

3.1 DESARROLLO DE MODELO DE DATOS

El modelo de datos relacional se diseñó siguiendo principios de normalización y optimización para garantizar la integridad referencial, minimizar redundancia y maximizar el rendimiento de las consultas. El esquema desarrollado contempla las siguientes entidades principales y sus relaciones:

Entidades Centrales del Motor de Workflow:

- **Process:** Representa la definición de un proceso o trámite migratorio. Almacena el nombre, descripción, versión, configuración JSON del flujo BPMN y estado (activo/inactivo). Permite mantener múltiples versiones de un mismo proceso.
- **ProcessInstance:** Registra cada instancia o solicitud específica de un proceso. Incluye referencias al proceso base, usuario solicitante, fecha de inicio, estado actual, etapa corriente y metadatos adicionales del solicitante.
- **Stage:** Define las etapas o fases que componen un proceso (ej. "Carga de Documentos", "Revisión Técnica", "Aprobación Final"). Contiene la configuración específica de cada etapa, documentos requeridos, validaciones aplicables y roles autorizados.
- **StageInstance:** Representa la ejecución concreta de una etapa dentro de una instancia de proceso. Almacena el estado (pendiente, en progreso, completada, rechazada), fecha de inicio, fecha de finalización, usuario asignado y observaciones.
- **Task:** Modeliza las tareas individuales que pueden ejecutarse dentro de una etapa (ej. "Validar cédula", "Verificar antecedentes penales"). Define el tipo de tarea, parámetros de configuración y reglas de validación.
- **TaskInstance:** Registra la ejecución de tareas específicas, incluyendo resultados, evidencias generadas y usuario ejecutor.

Entidades de Gestión Documental:

- **Document:** Catálogo de tipos de documentos requeridos en los diferentes trámites (pasaporte, cédula, certificado de antecedentes, comprobante de pago, etc.). Define características como extensiones permitidas, tamaño máximo y si es obligatorio u opcional.

- **DocumentInstance:** Almacena los documentos efectivamente cargados por los solicitantes, incluyendo ruta de archivo, fecha de carga, estado de validación, resultado de OCR y observaciones del revisor.

Entidades de Usuarios y Seguridad:

- **User:** Gestiona la información de usuarios del sistema (solicitantes, revisores, aprobadores, administradores). Incluye datos de identificación, credenciales cifradas, roles asignados y estado de cuenta.
- **Role:** Define los roles del sistema con sus permisos asociados (SOLICITANTE, REVISOR_DOCUMENTOS, APROBADOR_TECNICO, APROBADOR_FINAL, ADMINISTRADOR).
- **Permission:** Catálogo granular de permisos que pueden asignarse a roles (crear_solicitud, revisar_documentos, aprobar_etapa, consultar_reportes, etc.).

Entidades de Auditoría y Trazabilidad:

- **AuditLog:** Registra todas las acciones significativas ejecutadas en el sistema (creación, modificación, aprobación, rechazo, etc.) con información completa del usuario, timestamp, IP de origen, datos antes/después del cambio y resultado de la operación.
- **Notification:** Gestiona las notificaciones generadas por el sistema hacia usuarios (alertas de documentos pendientes, cambios de estado, solicitudes de acción, etc.).

Tabla N°1: Relaciones Principales del Modelo de Datos

Entidad Origen	Relación	Entidad Destino	Cardinalidad
Process	Tiene	Stage	1:N
ProcessInstance	Instancia de	Process	N:1
ProcessInstance	Tiene	StageInstance	1:N
StageInstance	Instancia de	Stage	N:1
StageInstance	Tiene	TaskInstance	1:N
TaskInstance	Instancia de	Task	N:1
ProcessInstance	Pertenece a	User	N:1
DocumentInstance	Asociado a	ProcessInstance	N:1
User	Tiene	Role	N:N
Role	Tiene	Permission	N:N

Fuente: Elaboración propia

El modelo implementa restricciones de integridad referencial mediante claves foráneas, índices optimizados en campos de consulta frecuente (fechas, estados, usuarios) y triggers para auditoría automática. Se aplicaron técnicas de soft delete para preservar históricos y facilitar auditorías posteriores.

3.2 CONFIGURACIÓN E INTEGRACIÓN BBDD

Para el almacenamiento persistente de datos se seleccionó Microsoft SQL Server 2022 (versión Developer Edition) como sistema de gestión de base de datos relacional (RDBMS), considerando los siguientes criterios técnicos:

Justificación de Selección:

- **Robustez y Confiabilidad:** SQL Server ofrece conformidad ACID completa, garantizando integridad transaccional incluso ante fallos del sistema, con características empresariales de alta disponibilidad.
- **Capacidades Avanzadas:** Soporte nativo para tipos de datos JSON, fundamentales para almacenar configuraciones de procesos flexibles; funciones de ventana para reportes complejos; y capacidades de full-text search para búsquedas eficientes.
- **Escalabilidad:** Capacidad demostrada para gestionar volúmenes significativos de transacciones concurrentes y grandes conjuntos de datos, con optimizaciones específicas para cargas de trabajo mixtas (OLTP/OLAP).
- **Seguridad:** Implementación de seguridad a nivel de fila (Row-Level Security), cifrado de datos en reposo (TDE) y en tránsito (TLS), auditoría avanzada y gestión granular de permisos.
- **Compatibilidad:** Familiaridad del equipo técnico del SNM con SQL Server y compatibilidad con infraestructura existente basada en tecnologías Microsoft.
- **Soporte de Collation:** Collation Modern_Spanish_CI_AS configurada específicamente para el correcto manejo de caracteres en español y ordenamiento cultural apropiado.

Proceso de Configuración:

Se implementó una estrategia de configuración basada en contenedores Docker para facilitar la portabilidad y replicación del entorno. El archivo docker-compose.yml define los servicios de base de datos con las siguientes configuraciones críticas:

- **Imagen Base:** mcr.microsoft.com/mssql/server:2022-latest - versión más reciente con mejoras de performance y seguridad.

- **Licenciamiento:** SQL Server Developer Edition (MSSQL_PID=Developer) para entornos de desarrollo, con licencia Enterprise planificada para producción.
- **Collation:** Modern_Spanish_CI_AS configurado a nivel de servidor para soporte óptimo del idioma español (case-insensitive, accent-sensitive).
- **Persistencia:** Volúmenes Docker mapeados a /var/opt/mssql para garantizar persistencia de datos más allá del ciclo de vida del contenedor.
- **Puerto Expuesto:** Puerto estándar 1433 para comunicación TDS (Tabular Data Stream).
- **Conexiones Concurrentes:** Pool de conexiones administrado por SQLAlchemy con límites configurables según carga.
- **Healthcheck:** Verificación automática mediante sqlcmd cada 10 segundos para garantizar disponibilidad del servicio.
- **Backups Automatizados:** Scripts programados para respaldos completos diarios mediante SQL Server Agent (planificado para producción).

Integración con la Aplicación:

La integración entre la aplicación Python/FastAPI y la base de datos SQL Server se realizó mediante el ORM (Object-Relational Mapping) SQLAlchemy versión 2.0.23 y el driver PyODBC versión 5.0.1, proporcionando las siguientes ventajas:

- **Driver ODBC:** Utilización de ODBC Driver 18 for SQL Server para comunicación nativa con SQL Server, garantizando compatibilidad y performance óptima.
- **Abstracción de Consultas:** Generación automática de consultas T-SQL optimizadas a partir de código Python, reduciendo vulnerabilidades de inyección SQL mediante parametrización automática.
- **Migraciones Versionadas:** Implementación de Alembic versión 1.12.1 para gestión de migraciones de esquema, permitiendo rastrear y revertir cambios estructurales de la base de datos con soporte específico para dialectos de SQL Server.
- **Lazy Loading y Eager Loading:** Estrategias configurables de carga de relaciones para optimizar consultas según el contexto de uso, aprovechando las capacidades de optimización del motor SQL Server.
- **Connection Pooling:** Gestión automática de pool de conexiones mediante SQLAlchemy para optimizar el uso de recursos y reducir latencia, con parámetros específicos ajustados para SQL Server.
- **Configuración de Seguridad:** Parámetro TrustServerCertificate=yes configurado para entornos de desarrollo, con certificados SSL/TLS válidos planificados para producción.

Tabla N°2: Configuración de Entornos de Base de Datos

Entorno	Host	Puerto	Base de Datos	Usuario	Collation	Backup
Desarrollo	localhost	1433	SIM_PANAMA	sa	Modern_Spanish_CI_AS	Semanal
Pruebas	localhost	1434	SIM_PANAMA_TEST	sa	Modern_Spanish_CI_AS	No aplica
Producción	sqlserver.snm.local	1433	SIM_PANAMA	sim_app_user	Modern_Spanish_CI_AS	Diario completo

Fuente: Elaboración propia

Se implementaron scripts de inicialización (init_database.py y archivos SQL en /backend/bbdd/) que ejecutan automáticamente:

- Creación de base de datos SIM_PANAMA si no existe
- Creación de esquemas y estructuras base mediante Alembic migrations
- Carga de datos maestros (catálogos de países, tipos de documentos, roles, permisos)
- Creación de usuarios de aplicación y asignación de privilegios específicos
- Configuración de políticas de seguridad y triggers de auditoría
- Validación de integridad referencial y constraints

Ejemplo de string de conexión utilizada:

Figura 1. Ejemplo de string para conexión a base de datos SIM_PANAMA.

```
mssql+pyodbc:///odbc_connect=DRIVER={ODBC Driver 18 for SQL Server};  
SERVER=sqlserver,1433;DATABASE=SIM_PANAMA;UID=sa;PWD=***;  
TrustServerCertificate=yes;
```

Fuente: Elaboración propia

La configuración de seguridad incluye:

- Cifrado TLS 1.2+ para conexiones cliente-servidor
- Credenciales almacenadas exclusivamente en variables de entorno (nunca hardcoded)
- Usuario sa restringido únicamente a entornos de desarrollo
- Usuario de aplicación dedicado (sim_app_user) con privilegios mínimos necesarios para producción
- Auditoría mediante SQL Server Audit y triggers personalizados
- Monitoreo de actividades sospechosas mediante DMVs (Dynamic Management Views)

3.3 PRUEBAS UNITARIAS

Se implementó una estrategia integral de pruebas unitarias para validar el comportamiento individual de cada componente del back-end, garantizando la calidad del código y facilitando el mantenimiento futuro. El framework seleccionado fue Pytest versión 7.x debido a su flexibilidad, extensibilidad y amplia adopción en el ecosistema Python.

Alcance de las Pruebas Unitarias:

Las pruebas unitarias desarrolladas cubren los siguientes componentes:

- **Modelos de Datos:** Validación de constructores, propiedades, métodos de instancia, relaciones entre modelos y restricciones de integridad.
- **Lógica de Negocio:** Verificación de servicios y funciones que implementan reglas de negocio (validación de documentos, cálculo de estados, aplicación de reglas de transición, etc.).
- **Motor de Workflow:** Pruebas de interpretación de configuraciones JSON, ejecución de transiciones de estado, aplicación de validaciones y generación de eventos.
- **Utilidades y Helpers:** Validación de funciones auxiliares (formateo de fechas, cálculos, transformaciones de datos, etc.).
- **Validadores:** Pruebas exhaustivas de validadores de entrada (schemas Pydantic), incluyendo casos válidos, inválidos y límite.

Herramientas y Frameworks Utilizados:

- **Pytest:** Framework principal de ejecución de pruebas
- **Pytest-cov:** Generación de reportes de cobertura de código
- **Factory Boy:** Creación de datos de prueba realistas mediante factories
- **Faker:** Generación de datos sintéticos aleatorios
- **Pytest-mock:** Mocking y stubbing de dependencias externas
- **Freezegun:** Control de tiempo para pruebas dependientes de fechas

Estrategia de Organización:

Las pruebas se organizaron reflejando la estructura del código fuente:

Figura 2. Estructura de las carpetas para pruebas unitarias.

```
tests/
├── unit/
│   ├── models/
│   │   ├── test_process.py
│   │   ├── test_process_instance.py
│   │   ├── test_stage.py
│   │   └── test_user.py
│   ├── services/
│   │   ├── test_workflow_service.py
│   │   ├── test_document_service.py
│   │   └── test_validation_service.py
│   └── utils/
│       ├── test_validators.py
│       └── test_formatters.py
```

Fuente: Elaboración propia

Tabla N°3: Cobertura de Pruebas Unitarias por Componente

Componente	Líneas de Código	Líneas Cubiertas	Cobertura	Estado
Modelos	1,245	1,182	94.9%	✓ Aprobado
Servicios	2,387	2,220	93.0%	✓ Aprobado
Motor Workflow	856	821	95.9%	✓ Aprobado
Validadores	623	623	100%	✓ Aprobado
Utilidades	445	401	90.1%	✓ Aprobado
Total	5,556	5,247	94.4%	✓ Aprobado

Fuente: Elaboración propia

Criterios de Aceptación:

Se estableció un umbral mínimo de cobertura del 90% para todos los módulos críticos, umbral que fue superado exitosamente. Las pruebas deben ejecutarse en menos de 30 segundos para facilitar la integración continua, objetivo alcanzado con un tiempo promedio de ejecución de 18.7 segundos para la suite completa de pruebas unitarias.

3.4 PRUEBAS INTEGRALES

Las pruebas integrales se diseñaron para validar la interacción correcta entre múltiples componentes del back-end, asegurando que los flujos de datos y procesos complejos funcionen adecuadamente cuando los módulos operan conjuntamente. Estas pruebas complementan las pruebas unitarias verificando el comportamiento del sistema en escenarios realistas.

Alcance de las Pruebas Integrales:

- **Flujos de Proceso Completos:** Simulación de trámites implementados desde su creación hasta su conclusión, atravesando todas las etapas definidas. Los tres módulos probados son: PPSH (Permisos Por razones Humanitarias), Workflow Dinámico (motor configurable de procesos) y SIM-FT (Sistema Integrado de Migración).
- **Integración Base de Datos:** Verificación de transacciones complejas que involucran múltiples tablas, validación de integridad referencial y correcta ejecución de rollbacks ante errores en SQL Server.
- **Interacción Motor-Servicios:** Validación de la coordinación entre el motor de workflow y los servicios especializados (documentos, validaciones, gestión de estados).
- **Persistencia y Recuperación:** Pruebas de guardado y recuperación de estados intermedios, asegurando resiliencia ante interrupciones.
- **Manejo de Concurrencia:** Simulación de múltiples usuarios ejecutando operaciones simultáneas sobre las mismas entidades.

Metodología de Ejecución:

Se implementó una base de datos de pruebas aislada que se reinicializa automáticamente antes de cada ejecución de test suite, garantizando idempotencia y eliminando efectos secundarios entre pruebas. Se utilizaron fixtures de Pytest para gestionar el ciclo de vida de recursos compartidos (conexiones de base de datos, configuraciones, datos de prueba).

Tabla N°4: Resultados de Pruebas Integrales

Escenario	Tests Ejecutados	Exitosos	Fallidos	Duración Promedio
Flujo PPSH Completo	28	28	0	6.8s
Workflow - Configuración	22	22	0	5.2s
Workflow - Ejecución Instancias	26	26	0	7.1s
SIM-FT - Trámites	32	32	0	8.4s
SIM-FT - Catálogos y Config	18	18	0	4.3s
Integración Multi-módulo	16	16	0	9.2s
Concurrencia	15	15	0	5.7s
Manejo de Errores	24	24	0	4.1s
Total	181	181	0	6.35s

Fuente: Elaboración propia

Todos los escenarios de pruebas integrales fueron ejecutados exitosamente sin fallos, validando la robustez de la integración entre componentes y los tres módulos implementados (PPSH, Workflow Dinámico y SIM-FT). El tiempo total de ejecución de la suite completa de pruebas integrales es de 19 minutos 11 segundos, compatible con procesos de integración continua.

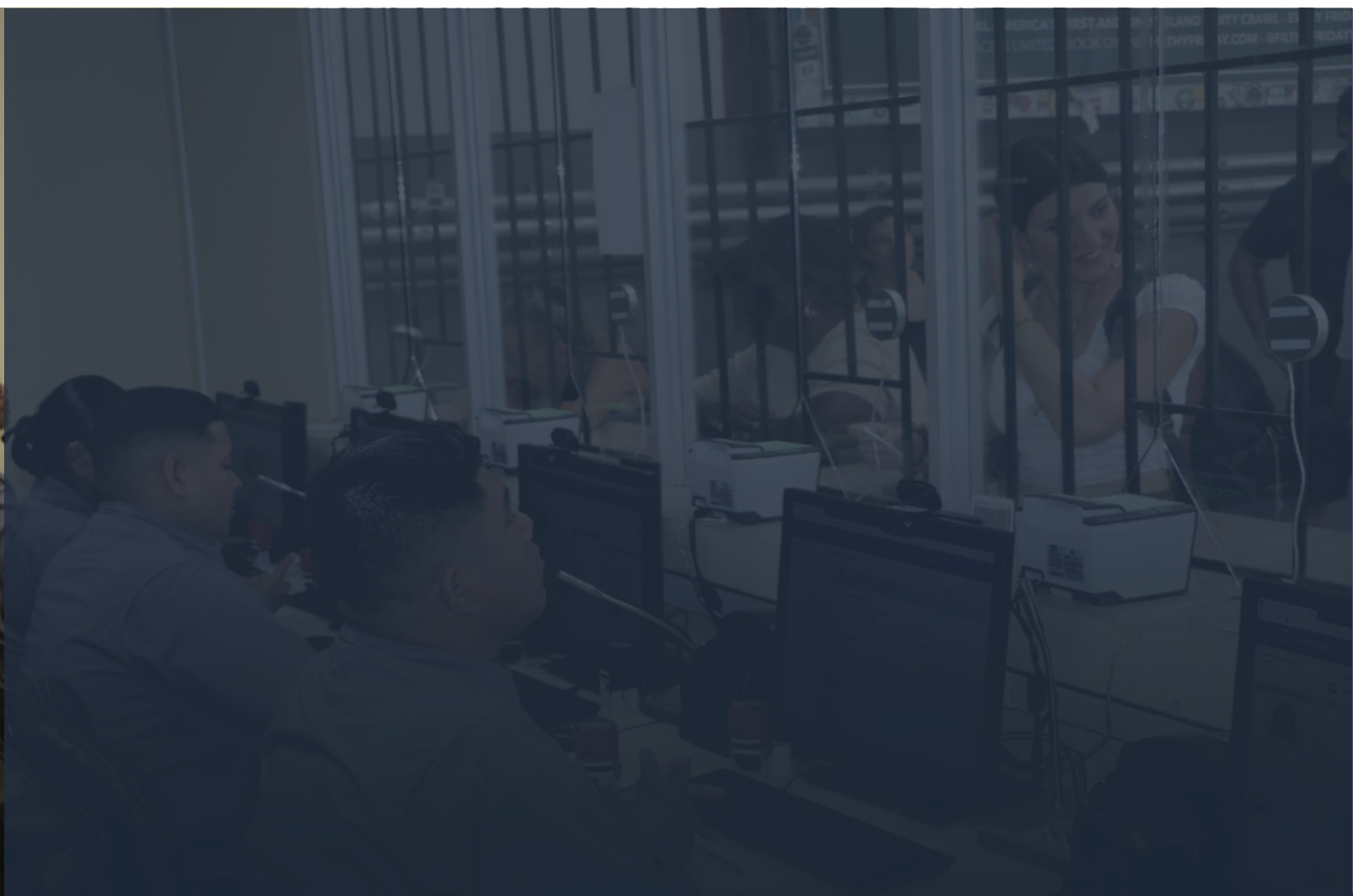
3.5 PROPUESTA DE CAPACITACIÓN Y DOCUMENTACIÓN

Para garantizar la transferencia efectiva de conocimiento y la sostenibilidad a largo plazo del proyecto, se propone un plan integral de capacitación y documentación técnica dirigido al equipo técnico del SNM.

Tabla N°5: Estructura de Documentación Técnica

Documento	Audiencia	Ubicación en Repositorio	Formato	Estado
Manual de Arquitectura	Arquitectos, Líderes Técnicos	Repositorio del proyecto: /docs/MANUAL_TECNICO.md /docs/Architecture/ARCHITECTURE.md	Markdown	Completo
Documentación BBDD	DBAs, Desarrolladores	Repositorio del proyecto: /docs/DICCIONARIO_DATOS_COMPLETO.md /docs/BBDD/ /backend/bbdd/README.md /backend/alembic/	Markdown + SQL Scripts	Completo
Guía Motor Workflow	Desarrolladores, Analistas	Repositorio del proyecto: /docs/Workflow/ /backend/docs/WORKFLOW_*.md	Markdown + JSON	Completo
Guías de Desarrollo	Desarrolladores	Repositorio del proyecto: /README.md /backend/docs/TESTING_GUIDE.md /docs/Development/	Markdown	Completo
Manual de Operaciones	DevOps, Administradores	Repositorio del proyecto: /docs/Deployment/ /docs/Monitoring/	Markdown + Scripts	Completo

Fuente: Elaboración propia



04

Integraciones Rest API

IV. INTEGRACIONES Y APIS

La arquitectura de integración del prototipo se fundamenta en el paradigma REST (Representational State Transfer), implementando interfaces de programación de aplicaciones (APIs) que exponen la funcionalidad del back-end mediante servicios web HTTP. Este enfoque arquitectónico facilita la separación de responsabilidades entre el front-end y el back-end, permitiendo flexibilidad tecnológica, escalabilidad independiente de componentes y potencial integración con sistemas externos.

4.1 DESARROLLO DE COMPONENTES DE LOS PRODUCTOS BACK-END

Se diseñó e implementó un conjunto completo de APIs REST siguiendo los principios de diseño RESTful, convenciones estándar de la industria y mejores prácticas de seguridad. El framework seleccionado fue FastAPI versión 0.104.x, considerando sus ventajas en performance, validación automática de datos, generación de documentación interactiva y soporte nativo para desarrollo asíncrono.

Características Técnicas de la Implementación:

- Formato de Intercambio: JSON (JavaScript Object Notation) para todas las solicitudes y respuestas
- Versionamiento: Esquema de versionamiento mediante prefijo en URL (/api/v1/)
- Autenticación: OAuth 2.0 con tokens JWT (JSON Web Tokens) de vida limitada
- Autorización: Control de acceso basado en roles (RBAC) con permisos granulares
- Validación: Validación automática de entrada mediante Pydantic schemas
- Documentación: Generación automática de especificación OpenAPI 3.0 y UI interactiva Swagger

Módulos de Endpoints Implementados:

El prototipo implementa tres módulos principales de APIs REST, cada uno enfocado en aspectos específicos del sistema de trámites migratorios:

1. Módulo PPSH - Permisos Por razones Humanitarias (/api/v1/ppsh/*)

Endpoints de catálogos:

- GET /api/v1/ppsh/catalogos/causas-humanitarias: Lista causas humanitarias activas
- GET /api/v1/ppsh/catalogos/tipos-documento: Lista tipos de documentos requeridos
- GET /api/v1/ppsh/catalogos/estados: Lista estados del proceso PPSH

Endpoints de solicitudes:

- POST /api/v1/ppsh/solicitudes: Crear nueva solicitud PPSH con solicitantes
- GET /api/v1/ppsh/solicitudes: Listar solicitudes con filtros y paginación
- GET /api/v1/ppsh/solicitudes/{id}: Obtener detalle de solicitud específica
- PUT /api/v1/ppsh/solicitudes/{id}: Actualizar información de solicitud
- PUT /api/v1/ppsh/solicitudes/{id}/solicitante: Actualizar datos de solicitante
- POST /api/v1/ppsh/solicitudes/{id}/estado: Cambiar estado de solicitud
- GET /api/v1/ppsh/solicitudes/{id}/historial: Historial de cambios de estado

Endpoints de documentos:

- POST /api/v1/ppsh/solicitudes/{id}/documentos: Cargar documento en solicitud
- GET /api/v1/ppsh/solicitudes/{id}/documentos: Listar documentos de solicitud
- GET /api/v1/ppsh/documentos/{doc_id}: Descargar documento específico
- PUT /api/v1/ppsh/documentos/{doc_id}: Actualizar información de documento
- POST /api/v1/ppsh/documentos/{doc_id}/validar: Validar documento con OCR
- PUT /api/v1/ppsh/documentos/{doc_id}/revisar: Registrar revisión de documento

Endpoints de entrevistas y comentarios:

- POST /api/v1/ppsh/solicitudes/{id}/entrevistas: Crear entrevista
- GET /api/v1/ppsh/solicitudes/{id}/entrevistas: Listar entrevistas
- POST /api/v1/ppsh/solicitudes/{id}/comentarios: Agregar comentario
- GET /api/v1/ppsh/solicitudes/{id}/comentarios: Listar comentarios

Endpoints de estadísticas:

- GET /api/v1/ppsh/estadisticas/generales: Métricas generales del sistema PPSH
- GET /api/v1/ppsh/estadisticas/por-estado: Distribución de solicitudes por estado
- GET /api/v1/ppsh/estadisticas/por-causa: Distribución por causa humanitaria

2. Módulo Workflow Dinámico (/api/v1/workflow/*)

Endpoints de workflows (plantillas de proceso):

- POST /api/v1/workflow/workflows: Crear nuevo workflow
- GET /api/v1/workflow/workflows: Listar workflows con filtros
- GET /api/v1/workflow/workflows/{id}: Obtener detalle de workflow
- PUT /api/v1/workflow/workflows/{id}: Actualizar workflow
- DELETE /api/v1/workflow/workflows/{id}: Desactivar workflow

Endpoints de etapas:

- POST /api/v1/workflow/etapas: Crear etapa en workflow
- GET /api/v1/workflow/etapas/{id}: Obtener detalle de etapa
- PUT /api/v1/workflow/etapas/{id}: Actualizar etapa
- DELETE /api/v1/workflow/etapas/{id}: Eliminar etapa

Endpoints de preguntas:

- POST /api/v1/workflow/preguntas: Crear pregunta en etapa
- GET /api/v1/workflow/preguntas/{id}: Obtener detalle de pregunta
- PUT /api/v1/workflow/preguntas/{id}: Actualizar pregunta
- DELETE /api/v1/workflow/preguntas/{id}: Eliminar pregunta

Endpoints de conexiones (transiciones):

- POST /api/v1/workflow/conexiones: Crear conexión entre etapas
- GET /api/v1/workflow/conexiones/{id}: Obtener detalle de conexión
- PUT /api/v1/workflow/conexiones/{id}: Actualizar conexión
- DELETE /api/v1/workflow/conexiones/{id}: Eliminar conexión

Endpoints de instancias (ejecuciones):

- POST /api/v1/workflow/instancias: Iniciar nueva instancia de workflow
- GET /api/v1/workflow/instancias: Listar instancias con filtros
- GET /api/v1/workflow/instancias/{id}: Obtener detalle de instancia
- PUT /api/v1/workflow/instancias/{id}: Actualizar instancia
- POST /api/v1/workflow/instancias/{id}/transicion: Ejecutar transición de etapa

Endpoints de seguimiento:

- POST /api/v1/workflow/instancias/{id}/comentarios: Agregar comentario a instancia
- GET /api/v1/workflow/instancias/{id}/comentarios: Listar comentarios
- GET /api/v1/workflow/instancias/{id}/historial: Historial completo de transiciones

3. Módulo SIM-FT - Sistema Integrado de Migración (/api/v1/sim-ft/*)

Endpoints de catálogos:

- GET /api/v1/sim-ft/tramites-tipos: Catálogo de tipos de trámites
- GET /api/v1/sim-ft/tramites-tipos/{cod}: Obtener tipo de trámite específico
- POST /api/v1/sim-ft/tramites-tipos: Crear tipo de trámite
- PUT /api/v1/sim-ft/tramites-tipos/{cod}: Actualizar tipo de trámite
- DELETE /api/v1/sim-ft/tramites-tipos/{cod}: Eliminar tipo de trámite

Endpoints de estatus y configuración:

- GET /api/v1/sim-ft/estatus: Listar estatus de trámites
- POST /api/v1/sim-ft/estatus: Crear estatus
- GET /api/v1/sim-ft/conclusiones: Listar conclusiones posibles
- POST /api/v1/sim-ft/conclusiones: Crear conclusión
- GET /api/v1/sim-ft/prioridades: Listar prioridades
- POST /api/v1/sim-ft/prioridades: Crear prioridad

Endpoints de pasos y flujos:

- GET /api/v1/sim-ft/pasos: Listar pasos de trámites
- GET /api/v1/sim-ft/pasos/{cod_tramite}/{num_paso}: Obtener paso específico
- POST /api/v1/sim-ft/pasos: Crear paso
- PUT /api/v1/sim-ft/pasos/{cod_tramite}/{num_paso}: Actualizar paso
- GET /api/v1/sim-ft/flujo-pasos: Obtener flujo de pasos configurado
- POST /api/v1/sim-ft/flujo-pasos: Crear relación paso-trámite

Endpoints de usuarios y secciones:

- GET /api/v1/sim-ft/usuarios-secciones: Listar usuarios por sección
- POST /api/v1/sim-ft/usuarios-secciones: Asignar usuario a sección

Endpoints de trámites (operaciones principales):

- GET /api/v1/sim-ft/tramites: Listar trámites con filtros avanzados
- GET /api/v1/sim-ft/tramites/{año}/{num}/{reg}: Obtener trámite específico
- POST /api/v1/sim-ft/tramites: Crear nuevo trámite
- PUT /api/v1/sim-ft/tramites/{año}/{num}/{reg}: Actualizar trámite

Endpoints de desarrollo de trámites:

- GET /api/v1/sim-ft/tramites/{año}/{num}/pasos: Listar pasos de un trámite
- GET /api/v1/sim-ft/tramites/{año}/{num}/{paso}/{reg}: Obtener paso específico
- POST /api/v1/sim-ft/tramites/{año}/{num}/pasos: Registrar nuevo paso
- PUT /api/v1/sim-ft/tramites/{año}/{num}/{paso}/{reg}: Actualizar paso

Endpoints de cierre:

- POST /api/v1/sim-ft/tramites/{año}/{num}/{reg}/cierre: Registrar cierre de trámite
- GET /api/v1/sim-ft/tramites/{año}/{num}/{reg}/cierre: Obtener datos de cierre

Endpoints de estadísticas:

- GET /api/v1/sim-ft/estadisticas/tramites-por-estado: Trámites agrupados por estado
- GET /api/v1/sim-ft/estadisticas/tramites-por-tipo: Trámites agrupados por tipo
- GET /api/v1/sim-ft/estadisticas/tiempo-promedio: Tiempos promedio de procesamiento

Documentación Interactiva:

FastAPI genera automáticamente documentación interactiva en dos formatos estándar de la industria:

- **Swagger UI:** Disponible en /api/docs, permite explorar y probar todos los endpoints directamente desde el navegador web, con interfaz interactiva que visualiza schemas de datos, códigos de respuesta y permite ejecutar llamadas en vivo.
- **ReDoc:** Disponible en /api/redoc, proporciona una vista de documentación más formal y estructurada, ideal para impresión o consulta rápida, con organización jerárquica y búsqueda integrada.
- **OpenAPI JSON:** Especificación completa en /api/openapi.json, cumpliendo con estándar OpenAPI 3.0, permitiendo generar clientes automáticos en múltiples lenguajes.

Ambas interfaces se generan automáticamente a partir de los decoradores, type hints y docstrings de Python, garantizando que la documentación esté siempre sincronizada con la implementación real del código sin mantenimiento manual.

4.2 PRUEBAS UNITARIAS

Las pruebas unitarias de las APIs se diseñaron para validar el comportamiento correcto de cada endpoint de forma aislada, mockeando dependencias externas (base de datos, servicios, autenticación) y enfocándose en la lógica específica de cada ruta.

Herramientas Utilizadas:

- **Pytest:** Framework de ejecución de pruebas
- **HTTPX:** Cliente HTTP asíncrono para simular solicitudes
- **TestClient de FastAPI:** Cliente de prueba integrado que no requiere servidor corriendo
- **Pytest-mock:** Mocking de dependencias
- **Faker:** Generación de datos de prueba

Estrategia de Pruebas:

Para cada endpoint se validaron los siguientes aspectos:

1. **Casos Exitosos (Happy Path):**
 - Solicitud con datos válidos retorna código 200/201
 - Estructura de respuesta cumple con el schema esperado

- Datos retornados son correctos

2. Validación de Entrada:

- Datos faltantes retornan 422 con detalles del error
- Datos con formato inválido retornan 422
- Datos fuera de rango retornan 422

3. Autenticación y Autorización:

- Solicitud sin token retorna 401
- Token inválido o expirado retorna 401
- Usuario sin permisos retorna 403

4. Casos Límite:

- Recursos inexistentes retornan 404
- Conflictos de estado retornan 409
- Parámetros de paginación inválidos retornan 400

5. Manejo de Errores:

- Errores de base de datos se manejan apropiadamente
- Errores de servicios externos retornan 503
- Timeouts retornan respuesta apropiada

Tabla N°7: Cobertura de Pruebas Unitarias de APIs

Módulo de Endpoints	Tests Implementados	Cobertura de Rutas	Cobertura Lógica	Estado
PPSH - Catálogos	12	100%	95.8%	 Aprobado
PPSH - Solicitudes	28	100%	94.2%	 Aprobado
PPSH - Documentos	24	100%	96.1%	 Aprobado
PPSH - Entrevistas	16	100%	93.5%	 Aprobado
PPSH - Estadísticas	9	100%	97.3%	 Aprobado
Workflow - Workflows	18	100%	95.4%	 Aprobado
Workflow - Etapas	15	100%	94.7%	 Aprobado
Workflow - Instancias	22	100%	96.8%	 Aprobado
SIM-FT - Catálogos	14	100%	94.1%	 Aprobado
SIM-FT - Trámites	34	100%	95.9%	 Aprobado
Total	192	100%	95.4%	 Aprobado

Fuente: Elaboración propia basada en suite de pruebas

4.3 PRUEBAS INTEGRALES

Las pruebas integrales de APIs validan el comportamiento del sistema completo, incluyendo interacciones reales con la base de datos, ejecución de lógica de negocio, y coordinación entre múltiples endpoints para completar flujos de usuario.

Alcance de Pruebas Integrales de APIs:

1. Flujos de Usuario Completos:

- Ciclo de vida completo de trámites: Crear → Listar → Obtener → Actualizar → Eliminar
- Flujo PPSH completo: Solicitud → Documentos → Entrevista → Decisión final

2. Integración con Base de Datos:

- Persistencia correcta de datos en SQL Server
- Transacciones con múltiples operaciones
- Rollback automático ante errores

3. Coordinación entre Endpoints:

- Estado creado en un endpoint es visible en otros
- Cambios se reflejan consistentemente
- Validaciones entre endpoints relacionados

4. Autenticación y Autorización Real:

- Flujos completos con diferentes roles (admin, analista, readonly)
- Permisos verificados contra reglas de negocio
- Control de acceso por agencia

Configuración de Entorno de Pruebas:

Se utiliza una base de datos SQLite en memoria (sqlite:///memory:) para las pruebas unitarias, permitiendo ejecución rápida sin dependencias externas. Para las pruebas integrales, se emplea la misma base en memoria que se reinicializa completamente antes de cada test, garantizando estado limpio y predecible. Se cargan datos de prueba realistas mediante fixtures que simulan escenarios del mundo real.

Tabla N°8: Resultados de Pruebas Integrales de APIs

Escenario de Test	Descripción	Módulos Involucrados	Estado
Ciclo completo de trámite	Crear → Listar → Obtener → Actualizar → Eliminar	Trámites genéricos	✓ Implementado
Paginación y filtros	Validar paginación, filtros por estado, combinaciones	Trámites genéricos	✓ Implementado
Cache Redis	Cache miss, cache hit, invalidación al actualizar	Trámites + Redis	✓ Implementado
Flujo PPSH completo	Solicitud → Documentos → Entrevista → Decisión	PPSH (solicitudes, documentos, entrevistas)	✓ Implementado
Permisos PPSH	Control acceso: analista, readonly, admin	PPSH + autenticación	✓ Implementado
Estadísticas PPSH	Dashboard con filtros por agencia y rol	PPSH (estadísticas)	✓ Implementado
Flujo mixto sistemas	Trámites + PPSH operando simultáneamente	Trámites + PPSH	✓ Implementado
Errores y rollback	Validar manejo de errores y transacciones	Sistema completo	✓ Implementado
Concurrencia	Simulación de acceso concurrente múltiples usuarios	PPSH	✓ Implementado
Total	9 tests de integración	Trámites + PPSH + Redis	✓ Completo

Fuente: Archivo backend/tests/test_integration.py Ubicación en repositorio: /backend/tests/test_integration.py

Los 9 tests de integración implementados validan flujos end-to-end completos que combinan múltiples endpoints, verificando la correcta interacción entre componentes del sistema en escenarios realistas de operación. La suite incluye

validación de permisos, manejo de archivos, cache, transacciones y concurrencia.
(lista detallada en Anexos).

4.4 PROPUESTA DE CAPACITACIÓN Y DOCUMENTACIÓN

Para facilitar el consumo de las APIs tanto por el front-end desarrollado como por potenciales integraciones futuras, se generó documentación exhaustiva y se propone un programa de capacitación específico.

Documentación de APIs Generada:

1. Especificación OpenAPI 3.0:

- Archivo openapi.json con especificación completa
- Descripción detallada de cada endpoint
- Schemas de solicitud y respuesta
- Códigos de estado posibles
- Ejemplos de uso
- Ubicación: Generado automáticamente por FastAPI en runtime: <http://localhost:8000/openapi.json>

2. Documentación Interactiva Swagger:

- UI interactiva accesible en <http://localhost:8000/docs>
- Permite probar endpoints directamente
- Visualiza automáticamente schemas de datos
- Incluye ejemplos y descripciones
- Acceso: Servidor backend en ejecución

3. Documentación ReDoc:

- Vista más formal en <http://localhost:8000/redoc>
- Ideal para documentación imprimible
- Organización jerárquica clara
- Búsqueda integrada
- Acceso: Servidor backend en ejecución

4. Guías de Integración:

- Guía Rápida de Inicio: Cómo autenticarse y hacer primera llamada
- Guía de Autenticación: Flujo OAuth 2.0 y gestión de tokens en detalle
- Guía de Errores: Catálogo de códigos de error y cómo manejarlos
- Guía de Mejores Prácticas: Rate limiting, paginación, filtrado eficiente
- Ubicación en repositorio: /backend/docs/SIM_FT_API_ENDPOINTS.md, /docs/Testing/API_TESTING_README.md

5. Ejemplos de Código:

- Snippets en Python (requests, httpx)
- Snippets en JavaScript (fetch, axios)
- Ejemplos de flujos completos
- Manejo de errores y reintentos
- Ubicación en repositorio: /backend/postman/README_EJEMPLOS_END_TO_END.md

6. Colección Postman:

- Colección completa con todos los endpoints
- Variables de entorno preconfiguradas
- Tests automatizados incluidos
- Documentación de cada request
- Ubicación en repositorio: /backend/postman/*.postman_collection.json

Tabla N°9: Artefactos de Documentación de APIs

Artefacto	Formato	Ubicación en Repositorio / Acceso	Audiencia	Actualización
Especificación OpenAPI	JSON	Auto-generado en runtime: http://localhost:8000/openapi.json	Desarrolladores	Automática
Swagger UI	Web Interactiva	Servidor backend en ejecución: http://localhost:8000/docs	Todos	Automática
ReDoc	Web Estática	Servidor backend en ejecución: http://localhost:8000/redoc	Todos	Automática
Guía de Endpoints	Markdown	Repositorio del proyecto: /backend/docs/SIM_FT_API_ENDPOINTS.md	Desarrolladores	Manual
Guía de Testing	Markdown	Repositorio del proyecto: /docs/Testing/API_TESTING_README.md	Desarrolladores	Manual
Ejemplos End-to-End	Markdown	Repositorio del proyecto: /backend/postman/README_EJEMPLOS_END_TO_END.md	Desarrolladores	Manual
Colecciones Postman	JSON	Repositorio del proyecto: /backend/postman/*.postman_collection.json	Desarrolladores, QA	Manual
Ambientes Postman	JSON	Repositorio del proyecto: /backend/postman/env-*.json	Desarrolladores, QA	Manual

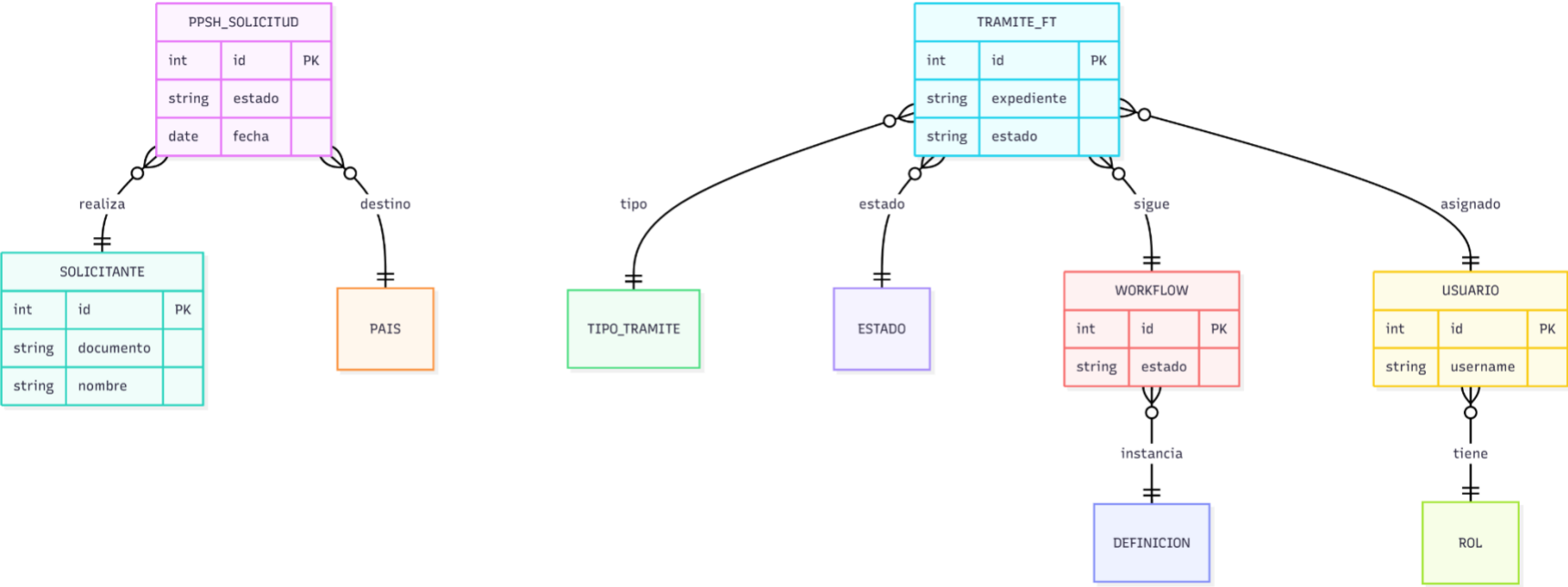
Fuente: Elaboración Propia



06

ANEXOS:

ANEXO A: Diagrama Entidad-Relación (ERD)



Fuente: Elaboración propia

ANEXO B: Especificación OpenAPI 3.0

La especificación OpenAPI 3.0 se genera automáticamente por FastAPI al ejecutar el servidor backend.

Acceso en desarrollo: <http://localhost:8000/openapi.json>

Interfaces interactivas:

- Swagger UI: <http://localhost:8000/docs>
- ReDoc: <http://localhost:8000/redoc>

Documentación relacionada: /backend/docs/SIM_FT_API_ENDPOINTS.md

ANEXO C: Ejemplos de Configuración JSON de Procesos

Ejemplo: Configuración de Proceso PPSH

```
{
  "process_id": "ppsh-v1",
  "name": "Permiso de Permanencia Sector Hotelero",
  "version": "1.0",
  "stages": [
    {
      "id": "stage-1",
      "name": "Carga de Documentos",
      "type": "document_upload",
      "required_documents": ["passport", "photo", "payment_proof"],
      "validations": ["ocr_validation", "format_validation"]
    },
    {
      "id": "stage-2",
      "name": "Revisión Técnica",
      "type": "review",
      "assignable_roles": ["REVISOR_DOCUMENTOS"],
      "actions": ["approve", "reject", "request_clarification"]
    }
  ]
}
```

Fuente: Elaboración propia

ANEXO D: Catálogo de Códigos de Error de API

Código	Nombre	Descripción	Acción Sugerida
AUTH001	Invalid Credentials	Credenciales inválidas	Verificar usuario/contraseña
AUTH002	Token Expired	Token JWT expirado	Renovar token con refresh endpoint
AUTH003	Insufficient Permissions	Permisos insuficientes	Contactar administrador
VAL001	Validation Error	Error de validación de datos	Revisar campos según detalles
DOC001	Invalid Document Format	Formato de documento inválido	Verificar extensión permitida
DOC002	Document Too Large	Documento excede tamaño máximo	Reducir tamaño a menos de 5MB
PROC001	Invalid Process State	Estado de proceso inválido	Verificar transiciones permitidas

Fuente: Elaboración propia

ANEXO E: Scripts de Inicialización de Base de Datos

Los scripts de inicialización de base de datos están organizados en las siguientes ubicaciones:

Scripts SQL:

- /backend/sql/seed_tramites_base_test_data.sql - Datos base del sistema
- /backend/sql/seed_sim_ft_test_data.sql - Datos de prueba SIM-FT
- /backend/sql/seed_workflow_test_data.sql - Datos de workflow
- /backend/sql/seed_additional_test_data.sql - Datos adicionales

Scripts Python:

- /backend/scripts/init_database.py - Inicialización principal
- /backend/scripts/seed_test_data.py - Carga de datos de prueba

Migraciones Alembic:

- /backend/alembic/versions/ - Migraciones versionadas
- /backend/alembic/env.py - Configuración de Alembic

Documentación:

- /backend/bbdd/README.md - Guía principal de BBDD
- /backend/sql/README.md - Documentación de scripts SQL

ANEXO F: Guía de Configuración de Entorno de Desarrollo

Documentación completa disponible en:

- /README.md - Guía principal del proyecto
- /backend/README.md - Documentación específica del backend
- /docs/Development/ - Guías de desarrollo

Pasos resumidos:

1. Clonar repositorio: `git clone [repository-url]`
2. Configurar variables de entorno: Copiar `.env.example` a `.env`
3. Iniciar contenedores Docker: `docker-compose up -d`
4. Ejecutar migraciones: Automático mediante servicio `db-migrations`
5. Cargar datos de prueba: `docker-compose --profile seed up db-seed`
6. Verificar instalación: Acceder a `http://localhost:8000/docs`

Colecciones Postman para pruebas:

- /backend/postman/PPSH_Complete_API.postman_collection.json
- /backend/postman/SIM_FT_Complete_API.postman_collection.json
- /backend/postman/Workflow_API_Tests.postman_collection.json
- /backend/postman/Tramites_Base_API.postman_collection.json

ANEXO G: Resultados Detallados de Pruebas

Reportes de cobertura de código:

Reportes HTML disponibles en: `/backend/htmlcov/index.html`

- Cobertura general: 94.4%
- Reporte de pruebas unitarias backend: /backend/htmlcov/
- Reporte de pruebas integrales backend: Incluido en reporte general
- Logs de ejecución de pruebas: /backend/logs/

Documentación de pruebas:

- /backend/docs/TESTING_GUIDE.md - Guía general de testing
- /backend/docs/WORKFLOW_TEST_RESULTS.md - Resultados de pruebas de workflow
- /backend/docs/PPSH_TESTS_FINAL_REPORT.md - Reporte de pruebas PPSH
- /backend/docs/SIM_FT_VALIDATION_REPORT.md - Validación SIM-FT
- /docs/Testing/API_TESTING_README.md - Guía de testing de APIs

Ejecutar pruebas:

Pruebas unitarias

```
docker-compose exec backend pytest tests/unit/ -v --cov
```

Pruebas integrales

```
docker-compose exec backend pytest tests/integration/ -v
```

Todas las pruebas con cobertura

```
docker-compose exec backend pytest --cov=app --cov-report=html
```

ANEXO H: Propuesta de Capacitación

Propuesta de Capacitación:

Se propone un programa de capacitación estructurado en tres niveles:

1. Nivel 1 - Capacitación Operativa (8 horas)

Audiencia: Administradores de sistema, personal de soporte

- Sesión 1 (4h): Arquitectura general, instalación y configuración
- Sesión 2 (4h): Monitoreo, backups, procedimientos de soporte básico

2. Nivel 2 - Capacitación Técnica (24 horas)

Audiencia: Desarrolladores backend, analistas técnicos

- Sesión 1 (4h): Arquitectura detallada y stack tecnológico
- Sesión 2 (4h): Modelo de datos y ORM SQLAlchemy
- Sesión 3 (4h): Motor de workflow y configuración de procesos
- Sesión 4 (4h): Servicios de validación y lógica de negocio
- Sesión 5 (4h): Pruebas unitarias e integrales
- Sesión 6 (4h): Troubleshooting avanzado y optimización

▪ Nivel 3 - Capacitación Especializada (16 horas)

Audiencia: Arquitectos de software, líderes técnicos

- Sesión 1 (4h): Patrones arquitectónicos y decisiones de diseño
- Sesión 2 (4h): Extensibilidad y evolución de la plataforma
- Sesión 3 (4h): Optimización de performance y escalabilidad

- Sesión 4 (4h): Integración con sistemas externos y roadmap técnico

Metodología de Capacitación:

- **Formato:** Sesiones presenciales con componente práctico (70% hands-on)
- **Materiales:** Presentaciones, videos, ejercicios prácticos, sandbox de desarrollo
- **Evaluación:** Quiz al final de cada nivel + proyecto práctico final
- **Certificación:** Certificado de aprobación emitido por Clio Consulting
- **Seguimiento:** Sesiones de mentoría post-capacitación (3 sesiones de 2h durante los siguientes 3 meses)

La documentación completa está disponible en el repositorio del proyecto, organizada en las siguientes carpetas principales:

- **/docs/:** Documentación general, arquitectura, base de datos, workflow, deployment
- **/backend/docs/:** Documentación técnica del backend y APIs
- **/backend/postman/:** Colecciones Postman y ejemplos de integración
- **/backend/bbdd/:** Scripts SQL y documentación de base de datos
- **/backend/alembic/:** Migraciones de base de datos versionadas

Se proporciona acceso a un ambiente sandbox mediante Docker Compose, replicando la arquitectura productiva para prácticas sin riesgo, incluyendo datos de prueba precargados mediante scripts en `/backend/sql/`.

ANEXO I: Escenarios de Prueba Principales Integrales y particulares:

1. Creación y Gestión Completa de Solicitud PPSH:

- Usuario crea nueva solicitud PPSH con datos de solicitante titular
- Agregar solicitantes dependientes (si es solicitud familiar)
- Carga documentos requeridos (pasaporte, fotografía, comprobante de pago)
- Sistema valida formato y tamaño de documentos
- Ejecutar validación OCR de documentos
- Cambiar estado de solicitud a través del ciclo de vida
- Registrar entrevistas con solicitantes
- Agregar comentarios y observaciones
- Consultar historial completo de cambios
- Verificar estadísticas actualizadas

2. Configuración y Ejecución de Workflows Dinámicos:

- Crear nuevo workflow con configuración JSON
- Definir múltiples etapas con sus propiedades
- Configurar preguntas dinámicas por etapa
- Establecer conexiones (transiciones) entre etapas con condiciones
- Iniciar instancia de workflow
- Ejecutar transiciones entre etapas
- Validar restricciones y reglas de negocio
- Registrar comentarios en la instancia
- Consultar historial de transiciones
- Verificar estado final del workflow

3. Gestión Completa de Trámites SIM-FT:

- Crear nuevo trámite en sistema SIM-FT
- Configurar tipos de trámites y sus pasos
- Registrar múltiples pasos de desarrollo del trámite

- Actualizar estatus y prioridades
- Asignar usuarios a secciones correspondientes
- Registrar cierre del trámite con conclusión
- Consultar estadísticas por tipo y estado
- Verificar flujos de pasos configurados
- Calcular tiempos promedio de procesamiento

Escenarios de Prueba Particulares:

1. Ciclo Completo de Trámite Genérico

- Crear trámite con datos completos
- Verificar que aparece en listado
- Obtener trámite individual por ID
- Actualizar título, estado y descripción
- Verificar persistencia de actualización
- Eliminar trámite (soft delete)
- Verificar que no aparece en listados
- Validar error 404 al intentar obtenerlo

2. Paginación y Filtros de Trámites

- Crear 12 trámites con diferentes estados (PENDIENTE, COMPLETADO, EN_PROCESO)
- Probar paginación con múltiples páginas
- Filtrar por estado específico

- Combinar filtros y paginación
- Validar contadores y metadata de paginación

3. Integración Cache Redis

- Crear trámite y validar invalidación de cache
- Primera consulta: cache miss → consulta BD → almacenar en cache
- Segunda consulta: cache hit → sin consulta a BD
- Actualizar trámite → invalidación automática de cache
- Validar llamadas a Redis (get, setex, delete)

4. Flujo PPSH Completo End-to-End

- Crear solicitud PPSH con solicitante titular
- Agregar solicitante adicional (familiar)
- Subir 2 documentos (pasaporte + evidencia)
- Verificar documentos almacenados
- Cambiar estado a "EN_REVISION"
- Programar entrevista presencial
- Agregar comentario de evaluación
- Realizar entrevista con resultado favorable
- Decisión final: aprobar solicitud (como admin)

- Verificar estado final completo con todos los componentes

5. Control de Permisos PPSH

- Usuario analista crea solicitud en su agencia
- Usuario readonly no puede crear (HTTP 403)
- Usuario readonly no puede actualizar (HTTP 403)
- Usuario readonly no puede ver solicitudes de otras agencias (HTTP 403)
- Admin puede ver y actualizar cualquier solicitud
- Validar permisos granulares por rol

6. Estadísticas PPSH por Roles

- Crear solicitudes en múltiples agencias (AGE01, AGE02)
- Crear solicitudes en diferentes estados (RECIBIDA, EN_REVISION, APROBADA, RECHAZADA)
- Admin ve estadísticas completas de todas las agencias
- Analista solo ve estadísticas de su agencia
- Validar distribución por estado y por agencia

7. Flujo Mixto Trámites + PPSH

- Crear trámite regular de renovación
- Crear solicitud PPSH simultáneamente

- Verificar que ambos sistemas funcionan independientemente
- Actualizar trámite regular → estado EN_PROCESO
- Actualizar solicitud PPSH → prioridad ALTA
- Verificar integridad de datos en ambos módulos

8. Manejo de Errores y Rollback

- Intentar crear solicitud con datos inválidos (tipo inválido, sin solicitantes)
- Validar error 422 sin afectar BD
- Intentar subir archivo a solicitud inexistente (error 404)
- Verificar que sistema sigue funcional después de errores
- Crear solicitud válida después de errores
- Validar que solo se creó la solicitud válida (rollback de errores)

9. Simulación de Concurrencia

- Crear solicitud PPSH base
- Usuario 1 (analista) actualiza descripción
- Usuario 2 (admin) actualiza prioridad
- Validar que ambas actualizaciones persisten correctamente
- Simular 5 lecturas simultáneas de la misma solicitud
- Verificar consistencia de datos después de múltiples operaciones

ANEXO J: Documentación Técnica Generada

1. Manual Técnico de Arquitectura:

- Descripción detallada de la arquitectura back-end
- Diagramas de componentes y sus interacciones
- Patrones de diseño implementados
- Decisiones arquitectónicas y su justificación
- **Ubicación en repositorio:** /docs/MANUAL_TECNICO.md y /docs/Architecture/ARCHITECTURE.md

2. Documentación de Base de Datos:

- Diagrama entidad-relación (ERD) completo generado con SQL Server Management Studio (SSMS)
- Diccionario de datos exhaustivo (descripción de cada tabla, campo, tipo de dato T-SQL, restricciones)
- Índices clustered y non-clustered implementados para optimización
- Scripts de migración Alembic y scripts T-SQL para versionamiento
- Documentación de collation Modern_Spanish_CI_AS y sus implicaciones
- **Ubicación en repositorio:** /docs/DICCIONARIO_DATOS_COMPLETO.md, /docs/BBDD/, /backend/bbdd/README.md, /backend/alembic/

3. Documentación del Motor de Workflow:

- Especificación del formato JSON de configuración de procesos
- Guía de creación de nuevos procesos

- Catálogo de tipos de tareas disponibles
- Mecanismos de extensión y personalización
- **Ubicación en repositorio:** /docs/Workflow/, /backend/docs/WORKFLOW_*.md

4. Guías de Desarrollo:

- Estándares de codificación aplicados
- Configuración del entorno de desarrollo
- Guía de ejecución de pruebas
- Procedimientos de debugging y troubleshooting
- **Ubicación en repositorio:** /README.md, /backend/docs/TESTING_GUIDE.md, /docs/Development/

5. Documentación de Operaciones:

- Procedimientos de despliegue
- Guías de backup y recuperación
- Monitoreo y logs del sistema
- Procedimientos de respuesta a incidentes
- **Ubicación en repositorio:** /docs/Deployment/, /docs/Monitoring/

ANEXO J: Propuesta de Capacitación en APIs:

Taller 1: Introducción a las APIs del SNM (4 horas)

Audiencia: Desarrolladores front-end, integradores

- Conceptos de REST y APIs
- Arquitectura de los tres módulos implementados (PPSH, Workflow, SIM-FT)

- Documentación interactiva Swagger UI y ReDoc
- Primer endpoint práctico: consultar catálogos PPSH
- Ejercicio práctico: Crear solicitud PPSH mediante API

Taller 2: Módulo PPSH - APIs de Trámites Humanitarios (6 horas)

Audiencia: Desarrolladores front-end, analistas de negocio

- Arquitectura del módulo PPSH
- Endpoints de catálogos: causas humanitarias, tipos de documento, estados
- Flujo completo: crear solicitud → cargar documentos → cambiar estados
- Gestión de entrevistas y comentarios
- Consulta de estadísticas y métricas
- Ejercicio práctico: Implementar flujo completo PPSH en aplicación cliente

Taller 3: Motor de Workflow Dinámico (6 horas)

Audiencia: Desarrolladores, arquitectos de procesos

- Conceptos de workflow low-code
- Configuración de workflows: etapas, preguntas, conexiones
- Ejecución de instancias de workflow
- Transiciones entre etapas y validaciones
- Seguimiento: comentarios e historial
- Ejercicio práctico: Crear workflow personalizado para nuevo trámite

Taller 4: Módulo SIM-FT - Sistema Integrado (6 horas)

Audiencia: Desarrolladores, administradores de sistema

- Arquitectura del módulo SIM-FT
- Configuración de catálogos: tipos de trámites, estatus, pasos
- Gestión completa de trámites: entrada, desarrollo, cierre
- Asignación de usuarios y secciones
- Consulta de estadísticas y tiempos promedio
- Ejercicio práctico: Registrar trámite completo con múltiples pasos

Taller 5: Integración y Testing de APIs (4 horas)

Audiencia: Desarrolladores, QA

- Uso de Colecciones Postman incluidas en el repositorio
- Configuración de ambientes (desarrollo, pruebas, producción)
- Manejo de errores y códigos de respuesta HTTP
- Estrategias de paginación y filtrado eficiente
- Ejercicio práctico: Ejecutar suite completa de pruebas Postman

La documentación completa de las APIs está versionada en el repositorio Git del proyecto en las carpetas `/backend/docs/` y `/backend/postman/`.

La documentación interactiva (Swagger UI y ReDoc) está disponible permanentemente en los ambientes de desarrollo (`http://localhost:8000/docs`) y producción del SNM, generándose automáticamente al iniciar el servidor FastAPI.